

EV Challenge Team Report

1. Team Information

- **Team Name:** Team Evolution

- **Team Members:**

- Akaash S S
- Om Jokhir Athitya
- Sonavadekar Rajvardhan Sarjerao
- Aditya Kumar
- Anjali Yadav

- **Mentor Name & Affiliation:**

Dr. Poonam Kumari

Professor, Department of Mechanical Engineering

Indian Institute of Technology Guwahati

2. FMU Details

- **FMU Created:** Yes

- **If Yes, Provide Details:**

This FMU is crafted to optimize vehicle speed in real-time, with the goal of boosting driving efficiency while staying within powertrain limits. It features four main logic modules that work together to smartly manage speed and calculate the necessary motor torque target:

Battery Current-Based Velocity Adjustment:

This module keeps an eye on the battery current and checks it against a set threshold. If the current goes over this limit, suggesting that the vehicle is pulling too much power and needs more torque, the system steps in to lower the optimal speed by a specific amount. If the current stays below the threshold, the vehicle maintains its default optimal speed.

State of Charge (SoC) Safety Control:

To protect battery life and avoid deep discharge, a second control logic looks at the battery's State of Charge. When the SoC dips below a certain level, the system activates a low-power mode by further decreasing the optimal speed, allowing the vehicle to run more conservatively.

Velocity Range Enforcement via Saturation Block:

The speed output is processed through a saturation block that keeps it within a set efficient operating range. This ensures the vehicle doesn't try to speed up or slow down beyond speeds that are both energetically optimal and structurally safe.

Torque Target Calculation Module:

The last logic block figures out the necessary motor torque based on the difference between actual and target speed, considering factors like vehicle weight, wheel size, gear ratio, and braking status. The calculated torque is then sent back to the CarMaker environment to manage the vehicle in a closed-loop simulation.

- **FMU Generation Method:**

The FMU was developed using MATLAB and implemented primarily through visual modeling in Simulink, leveraging its block-based architecture for algorithm design and integration.

3. FMU Interface and Parameters

- Number of Interfaces Added: 8

Signal Name	Link Type	Signal Source / Destination
Act_Vel	DDict	Car.v
Batt_Current	DDict	PT.BattV.Current
BrakePedal	DDict	DM.Brake
Cur_SoC	DDict	PT.BCU.BattHV.SOC
gear_ratio	DDict	PT.Gearbox.0.i_TrqIn2Out
regen_torque_limit	DDict	PT.WRL.Trq_BrakeReg_max
wheel_radius	DDict	Car.WRL.Radius
Motor_Trq_Target	DDict	PT.Control.Motor.Trq_trg

- **Parameters Used in FMU:**

Signal Name	Value
Batt_Current_Threshold	250 A
Batt_Current_V_Reduction	1.37 m/s
SoC_Threshold	0.3
SoC_V_Reduction	1.38 m/s
V_Max	12 m/s
V_Min	5 m/s
V_Optimal	9.722 m/s
Prop_Gain_Acc	0.4
max_motor_torque	450 Nm
vehicle_mass	2065 kg

4. Simulation Strategy

- **Approach to Choosing Optimal EV Speed:**

The ideal speed for electric vehicles (EVs) was chosen by finding a sweet spot between energy efficiency and real-world drivability. This speed falls within a range where the electric motor runs at peak efficiency while keeping battery drain to a minimum. Here are the key factors that were taken into account:

- The motor efficiency curve (which typically peaks at mid-range speeds)
- Common driving conditions in urban and suburban areas
- Vehicle limitations (like gear ratios and motor torque limits)

After considering all these elements, we determined that the best cruising speed is around 35 km/h (or about 9.7 m/s). This speed is found to be the most optimal, ensuring low power consumption without sacrificing performance.

• Observations During Simulation Execution:

During the simulation, the vehicle adapted quickly to changes in velocity error, battery current, and State of Charge. At first, the FMU didn't behave as we had anticipated since it was new, but after some adjustments, we managed to tweak the target velocity in real-time based on the thresholds we set. This helped us reduce speed when the current draw was high or the SoC was low. The torque output closely matched the calculated velocity error, showing that our proportional control was working effectively. The transitions between normal and reduced power modes were seamless, keeping the vehicle within the efficient speed range we defined. Overall, the simulation validated the logic flow and control strategies implemented, with no major anomalies or instabilities observed.

5. Team Experience and Learning

• Team's Experience in This EV Challenge:

Taking part in the EV Challenge was an incredibly rewarding and collaborative experience for our team. It gave us a hands-on opportunity to put our theoretical knowledge of electric vehicle dynamics, control systems, and energy management into practice. We tackled everything from crafting optimal velocity control strategies to seamlessly integrating Simulink models with IPG CarMaker using FMUs. This challenge helped us to sharpen our technical skills and enhance our problem-solving abilities. We walked away with valuable insights into real-time vehicle simulation, system modeling, and the critical role of efficient energy use in EVs. It was a fulfilling journey that deepened our understanding of EV technologies and strengthened our teamwork and engineering skills.

6. Optional Attachments

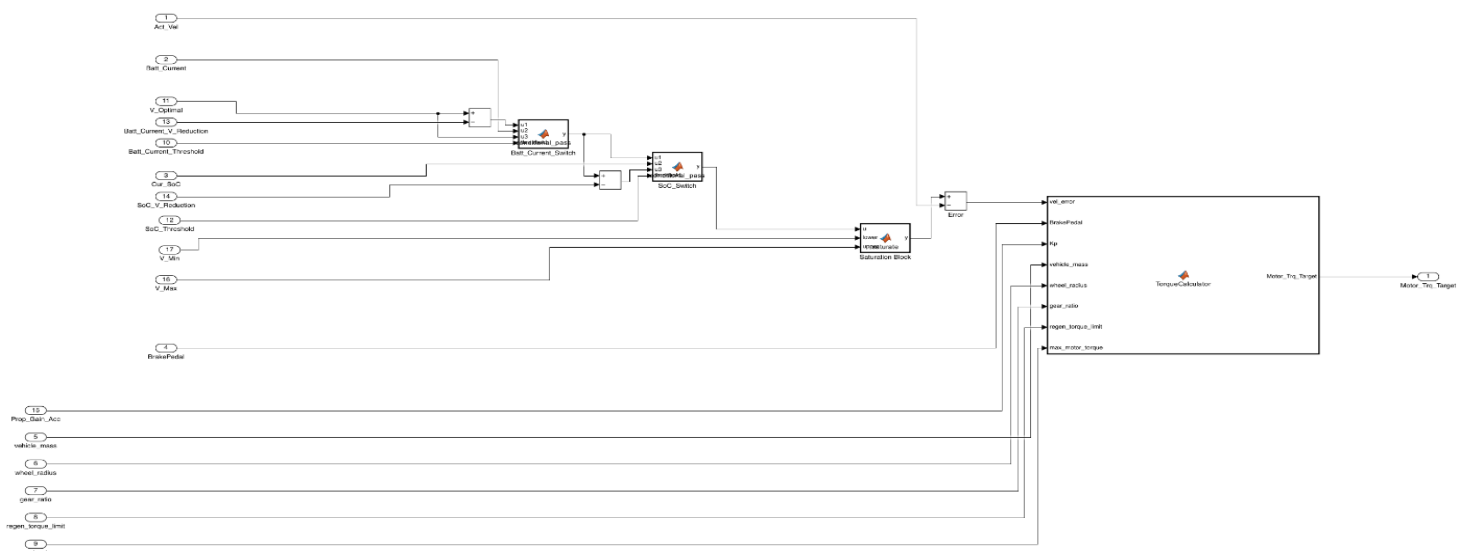


Fig 1. FMU Block Diagram

 MATLAB Function

```
function Motor_Trq_Target = TorqueCalculator(...  
    vel_error, BrakePedal, Kp, vehicle_mass, wheel_radius, ...  
    gear_ratio, regen_torque_limit, max_motor_torque)  
  
    desired_accel = Kp * vel_error;  
    required_force = vehicle_mass * desired_accel;  
    torque_at_wheel = required_force * wheel_radius;  
    motor_torque = torque_at_wheel / gear_ratio;  
  
    if (BrakePedal > 0) || (vel_error < 0)  
        if BrakePedal > 1  
            BrakePedal = BrakePedal / 100;  
        end  
  
        regen_torque = BrakePedal * regen_torque_limit;  
        Motor_Trq_Target = max(motor_torque, regen_torque);  
    else  
        Motor_Trq_Target = motor_torque;  
    end  
  
    Motor_Trq_Target = min(max(Motor_Trq_Target, regen_torque_limit), max_motor_torque);  
  
end
```

Fig 2. Torque Calculation Function

Submitted on: 27/06/2025

Submitted by: Akaash S S, Team Evolution